The integrated dashboard in MISP 2.5

Build a useful view, understand its scope

CIRCL / MISP Project

https://www.misp-project.org/

Reference snapshot: 29 October 2026

The integrated dashboard in MISP 2.5

Build a useful view, understand its scope

CIRCL / MISP Project

https://www.misp-project.org/

Reference snapshot: 29 October 2026

Build a view that supports a decision

1. Choose widgets for the question you need to answer.
2. Set their filters and understand their scope.
3. Save a useful personal layout.
4. Follow interesting results back to the underlying data.

Build a view that supports a decision
1. Choose widgets for the question you need to answer.
2. Set their filters and understand their scope.
3. Save a useful personal layout.
4. Follow interesting results back to the underlying data.

This course covers the integrated dashboard at /dashboards in the reviewed MISP 2.5.50 source. It does not imply every 2.5 patch level has the same interface. Screenshots are genuine browser captures of that source running locally with synthetic data, not mockups or production observations.

Two different dashboard applications

Surface	Integrated dashboard	Standalone misp-dashboard
Location	Inside MISP: `/dashboards`	Separate web application.
Model	Personal widgets, layouts and templates.	ZeroMQ activity stream and its own processing.
This training	Current standard course.	Historical course in the archive.

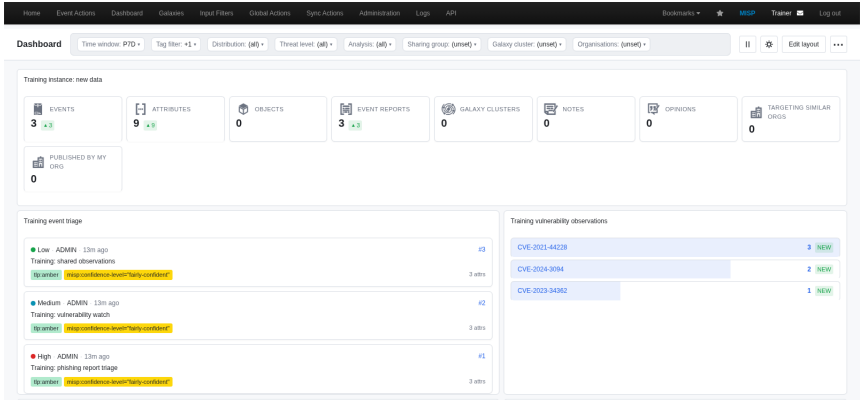
Two different dashboard applications

Surface	Integrated dashboard	Standalone misp-dashboard
Location	Inside MISP: `/dashboards`	Separate web application.
Model	Personal widgets, layouts and templates.	ZeroMQ activity stream and its own processing.
This training	Current standard course.	Historical course in the archive.

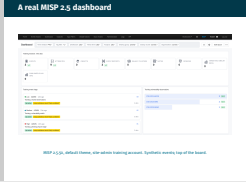
The former version of this deck taught the standalone Flask/Redis/ZeroMQ application. Its sources, screenshots and published PDF are archived. The integrated board does not require starting that application's subscriber, dispatcher or Flask server. Archiving its training says nothing about the standalone project's maintenance status.

Build your personal view

A real MISP 2.5 dashboard



MISP 2.5.50, default theme, site-admin training account. Synthetic events; top of the board.



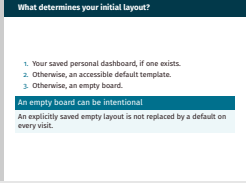
The overview contains new-data metrics, an Event Card Stream and Trending vulnerability identifiers. Two further tiles are below the crop. Counts are from three locally created events, not a real threat landscape. The standard interface retains its actual product styling in these captures.

What determines your initial layout?

1. Your saved personal dashboard, if one exists.
2. Otherwise, an accessible default template.
3. Otherwise, an empty board.

An empty board can be intentional

An explicitly saved empty layout is not replaced by a default on every visit.



The controller distinguishes no saved row from an empty array. Templates can be constrained by organisation, role and permission. Widget availability is checked separately. Do not promise every installation starts with the Analyst template: deployment state and template import matter.

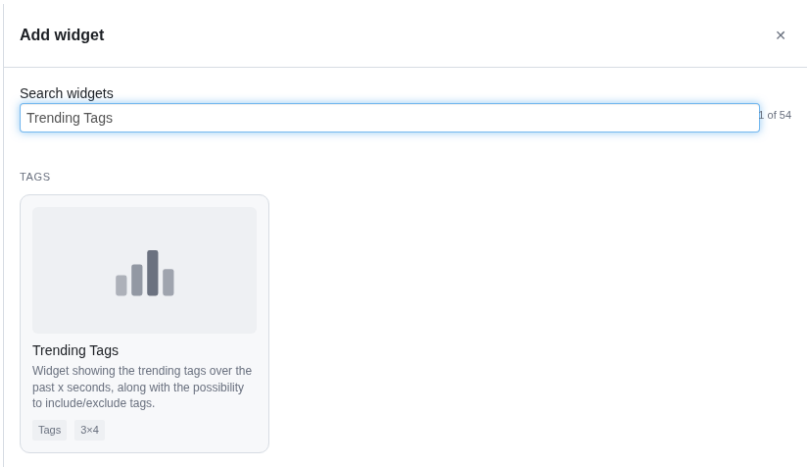
Layout edits and widget settings save differently

Task	Save behavior
Add, move, resize or remove tiles	Enter *Edit layout*; use *Save changes* or *Discard*.
Configure a widget	Its panel's *Save* persists configuration separately.
Apply a shared filter	Updates supporting widgets' configuration.

Discarding layout edits is not a general undo for saved widget settings.

The current JS stages geometry and membership in edit mode. Configuration and toolbar changes use per-widget saves rather than committing staged layout edits. A first unsaved board has a whole-layout fallback. Test on a saved personal board; export before making disruptive changes.

Find a widget in the gallery

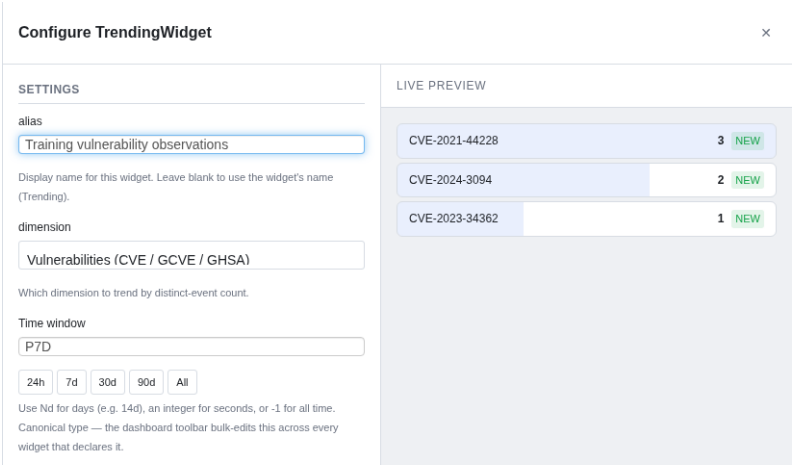


Actual gallery search for Trending Tags; the catalog is role-dependent.

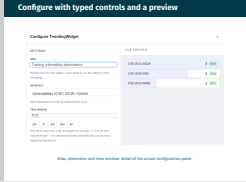


Enter Edit layout, then Add widget. Search by name or description, review the card and choose a widget. The gallery exposes only the classes available to this account. Its result count is an example, not a universal widget count. The preview glyph identifies the renderer, not live findings.

Configure with typed controls and a preview

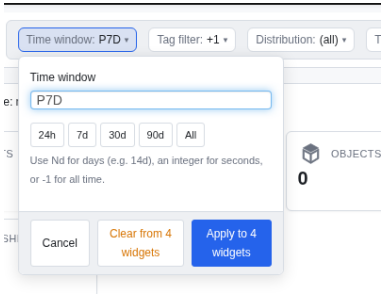


Alias, dimension and time window: detail of the actual configuration panel.

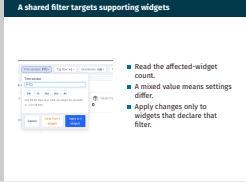


The panel includes an alias, a dimension selector, time window, threshold and minimum count, with a live preview. Advanced keys remain for parameters without a typed control. Save persists the widget configuration; Cancel closes without applying the form. The exercise uses min-count one to illustrate a tiny corpus, not as a production recommendation. This slide crops the panel for readability; the full PNG also shows the lower fields and Save/Cancel controls.

A shared filter targets supporting widgets



- Read the affected-widget count.
- A mixed value means settings differ.
- Apply changes only to widgets that declare that filter.



This five-tile board offers the time-window filter for four widgets. EventStreamCards inherits EventStream, which does not declare time-window in its schema here. Canonical types can have different schema keys, so the toolbar uses declared types rather than guessing from names. A shared time window is not a universal SQL filter over the entire page.

Make the scope visible to the next reader

- Give each widget a useful alias.
- Choose the time window and result limit deliberately.
- Check whether tags use exact labels or substring patterns.
- Use drill-down links to inspect the contributing data.

Make the scope visible to the next reader
- Give each widget a useful alias.
- Choose the time window and result limit deliberately.
- Check whether tags use exact labels or substring patterns.
- Use drill-down links to inspect the contributing data.

Widget contracts differ. Trending Tags uses include/exclude patterns; Event Stream's tags field remains a comma-separated string with negation. Do not copy one widget's configuration shape into every other widget. Aliases describe a tile; they do not change what its handler counts.

Interpret the numbers

Choose a widget for the question

Question	Useful starting point
What changed recently?	New data; event cards; recent reports or analyst data.
What recurs in the data?	Trending vulnerabilities, actors, techniques or tags.
Where is data associated?	Geolocation and country-map widgets.
Is the instance operating well?	Administrator health, resources and worker widgets.

Choose a widget for the question

Question | Useful starting point
What changed recently? | New data; event cards; recent reports or analyst data.
What recurs in the data? | Trending vulnerabilities, actors, techniques or tags.
Where is data associated? | Geolocation and country-map widgets.
Is the instance operating well? | Administrator health, resources and worker widgets.

Availability, permissions, configuration and prerequisites vary. For example, some monitoring and sighting widgets are site-admin-only in the reviewed implementation. A widget name alone does not establish that an analyst role can load it. Consult its description and handler.

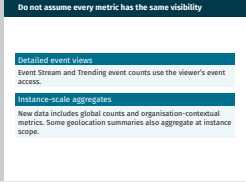
Do not assume every metric has the same visibility

Detailed event views

Event Stream and Trending event counts use the viewer's event access.

Instance-scale aggregates

New data includes global counts and organisation-contextual metrics. Some geolocation summaries also aggregate at instance scope.



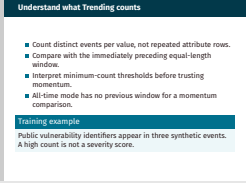
The NewDataStats handler deliberately has global scale counts for events, attributes and other data, plus separately contextual targeting and own-organisation publication metrics. AttributeGeoMap also has an aggregate posture. Do not claim all dashboard data is ACL-filtered like event detail. Review the actual widget and intended audience before sharing a board image.

Understand what Trending counts

- Count distinct events per value, not repeated attribute rows.
- Compare with the immediately preceding equal-length window.
- Interpret minimum-count thresholds before trusting momentum.
- All-time mode has no previous window for a momentum comparison.

Training example

Public vulnerability identifiers appear in three synthetic events.
A high count is not a severity score.



The example ranks vulnerability identifiers with counts three, two and one. NEW marks a value absent in the prior window. Trending's percentage momentum and the New-data widget's count deltas are different measures. The window follows data timestamps; it is not necessarily the event's descriptive date or when malicious activity occurred.

Refresh frequency is not data freshness

- Some widgets refresh automatically; others do not.
- Pause or resume automatic refresh from the header.
- Per-widget overrides are available where supported.
- A refresh may still return cached data.

Refresh frequency is not data freshness
- Some widgets refresh automatically; others do not.
- Pause or resume automatic refresh from the header.
- Per-widget overrides are available where supported.
- A refresh may still return cached data.

The scheduler uses the widget default or a saved refresh-delay override and pauses work on hidden pages. Zero disables an override's automatic refresh. WidgetCache and handler-specific caches are separate: a manual refresh is not a universal cache purge. The reviewed Trending cache is about twenty minutes; deployment and widget contracts determine freshness. No ZeroMQ dependency is implied by a timer.

Treat maps and coverage as evidence summaries

- Geolocation can be inferred from several kinds of data.
- A plotted origin is not proof of the attacker's location.
- An ATT&CK tile reflects tagged information.
- Tagged coverage is not proof of detection coverage.

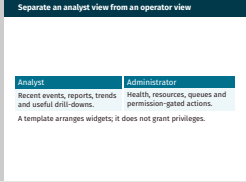
Treat maps and coverage as evidence summaries
- Geolocation can be inferred from several kinds of data.
- A plotted origin is not proof of the attacker's location.
- An ATT&CK tile reflects tagged information.
- Tagged coverage is not proof of detection coverage.

AttributeGeoMap's sources include IP, domain TLD, ASN and country/actor context; their precision differs. An animated map is a visualisation, not a replacement for evidential analysis. ATTACK views depend on the selected galaxy and the actual tags in scope. The reserved training names intentionally do not manufacture a real attack geography.

Separate an analyst view from an operator view

Analyst	Administrator
Recent events, reports, trends and useful drill-downs.	Health, resources, queues and permission-gated actions.

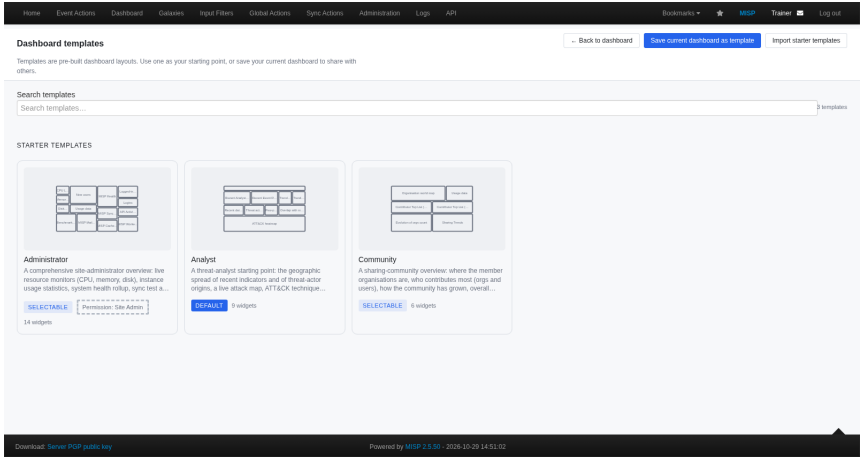
A template arranges widgets; it does not grant privileges.



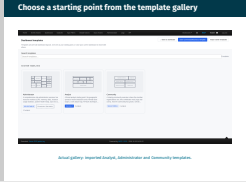
MispAdminHealth is site-admin-only and rolls up existing diagnostics. Some operator widgets expose actions, such as session invalidation; a dashboard is not automatically read-only merely because it displays metrics. Use an appropriate role and distinguish monitoring from making changes.

Reuse and verify

Choose a starting point from the template gallery



Actual gallery: imported Analyst, Administrator and Community templates.



The names and layout previews come from the shipped manifests in app/files/dashboard-templates. The screenshot account is a site administrator. Other users may see fewer templates or denied widget classes. Selecting a template is a personal layout reset, not a new data permission. A template preview is a layout miniature, not a screenshot of live results.

Reuse settings without confusing them with data

- Export your configuration before replacing a layout.
- Import a configuration or use an accessible template.
- Save a useful personal view as a template when permitted.
- Review identifiers and configuration before sharing it.

Reuse settings without confusing them with data
- Export your configuration before replacing a layout.
- Import a configuration or use an accessible template.
- Save a useful personal view as a template when permitted.
- Review identifiers and configuration before sharing it.

More contains Import configuration, Export configuration, Save as template and Browse templates. Built-in templates must be imported into the instance. Template restrictions affect availability; they do not expand event access. The server prefers a personal saved layout over a default. A saved template is not a live subscription that rewrites existing boards.

Configuration export and widget-data export differ

Configuration

Widget classes, options and positions for reusing a view.

Widget data

The result produced by a widget, available as JSON or flattened CSV.

Configuration export and widget-data export differ
Configuration
Widget classes, options and positions for reusing a view.
Widget data
The result produced by a widget, available as JSON or flattened CSV.

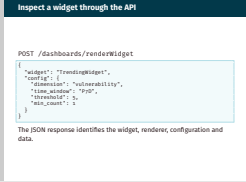
The portable layout is a widget array. The raw export endpoint retains a UserSetting envelope with its value; the UI normalises supported envelopes. A plain REST render returns a renderer/data envelope, while exportjson returns bare handler data. CSV flattens the result; it is not a universal MISP event export. Neither export form is automatically safe to share without review.

Inspect a widget through the API

```
POST /dashboards/renderWidget
```

```
{
  "widget": "TrendingWidget",
  "config": {
    "dimension": "vulnerability",
    "time_window": "P7D",
    "threshold": 5,
    "min_count": 1
  }
}
```

The JSON response identifies the widget, renderer, configuration and data.



This read-only request was executed against the actual local MISP instance using Accept: application/json. In the unchanged three-event lab, the returned Trending rows have distinct-event counts three, two and one. Do not embed a real API key in training material. Named exportjson:1 and exportcsv:1 select the raw-data export paths instead of the render envelope.

What is stored in a dashboard layout?

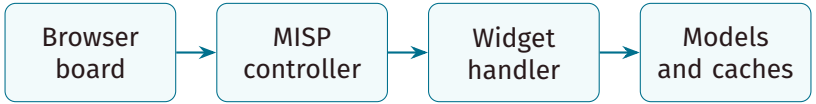
Field	Purpose
`instance_id`	Stable identifier for one tile.
`widget`	Widget class name.
`config`	Widget-specific settings and supported filters.
`position`	Grid coordinates and dimensions: `x`, `y`, `w`, `h`.

The supplied `examples/dashboard-layout.json` is a portable widget array.

What is stored in a dashboard layout?
Field Purpose
instance_id Stable identifier for one tile.
widget Widget class name.
config Widget-specific settings and supported filters.
position Grid coordinates and dimensions: x, y, w, h.
The supplied examples/dashboard-layout.json is a portable widget array.

Do not wrap that sample in an invented widgets property. The reviewed read-fixup normalises older position width/height keys to w/h and mints missing instance identifiers. The API updateSettings form instead accepts Dashboard.value; the import endpoint also accepts the array. Importing replaces the user's saved layout. Back up the old one first.

The integrated data path



Widget-specific prerequisites still apply.

The integrated data path
Browser board
MISP controller
Widget handler
Models and caches
Widget-specific prerequisites still apply.

The board calls the authenticated MISP controller; widget loading, schema adaptation and cache behavior are server-side concerns. Custom implementations belong in the existing Lib/Dashboard architecture. This replaces the former course's standalone subscriber/dispatcher diagram; it is not a claim that every widget needs no other dependency.

Diagnose the scope before rebuilding the board

Symptom	Check first
Widget missing	Class availability, user role and permissions.
Empty or different counts	Widget definition, time window, filters and data access.
Results seem old	Refresh state, timestamps and the widget's cache.
Controls do not work	Browser console/network, JavaScript assets and server errors.

Diagnose the scope before rebuilding the board

Symptom | Check first
Widget missing | Class availability, user role and permissions.
Empty or different counts | Widget definition, time window, filters and data access.
Results seem old | Refresh state, timestamps and the widget's cache.
Controls do not work. | Browser console/network, JavaScript assets and server errors.

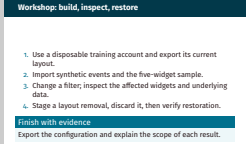
An empty result is not proof of no relevant threats. A hidden admin widget is not evidence that the data was deleted. Check the exact patch level and theme before comparing interface labels. Do not fix a personal view by blindly broadening instance-wide access.

Workshop: build, inspect, restore

1. Use a disposable training account and export its current layout.
2. Import synthetic events and the five-widget sample.
3. Change a filter; inspect the affected widgets and underlying data.
4. Stage a layout removal, discard it, then verify restoration.

Finish with evidence

Export the configuration and explain the scope of each result.



The README gives exact steps and sample files. The training events remain unpublished, organisation-only and non-IDS. No external enrichment or synchronisation is needed. Reports in the screenshots were created separately with synthetic Markdown; create a report during the workshop to populate that tile. Layout import changes this user's preferences.

What was verified for this course?

- Real MISP 2.5.50 browser screenshots with synthetic data.
- Layout import/export and widget rendering through the API.
- A staged removal and Discard in the actual browser.
- Slide and notes builds, examples and source references.

What was verified for this course?
- Real MISP 2.5.50 browser screenshots with synthetic data.
- Layout import/export and widget rendering through the API.
- A staged removal and Discard in the actual browser.
- Slide and notes builds, examples and source references.

Read the README for the exact validation record and limits. This is a local training-instance check, not the complete MISP test suite, an audit of every widget's permissions, or a production integration test. Screenshots do not prove a control works for every role or theme.

Sources and the next practical steps

- MISP 2.5 source and its shipped dashboard templates.
- This course's `README.md`: trainer guide, exercises and immutable source evidence.
- `images/misp25/README.md`: screenshot provenance.
- The standalone ZeroMQ course remains in the training archive.

Sources and the next practical steps
- MISP 2.5 source and its shipped dashboard templates.
- This course's README.md: trainer guide, exercises and immutable source evidence.
- images/misp25/README.md: screenshot provenance.
- The standalone ZeroMQ course remains in the training archive.

Continue with taxonomy/tagging and the MISP data-model course to interpret context, or inspect current widget contracts for development. The current source is authoritative; older developer examples and design handoff notes can describe an earlier stage of the dashboard.